Uber
dynamic pricing and ride demand

Time series, clustering, regression and a pricing loop, explained step by step

InnoIT · ML use case

Innoit

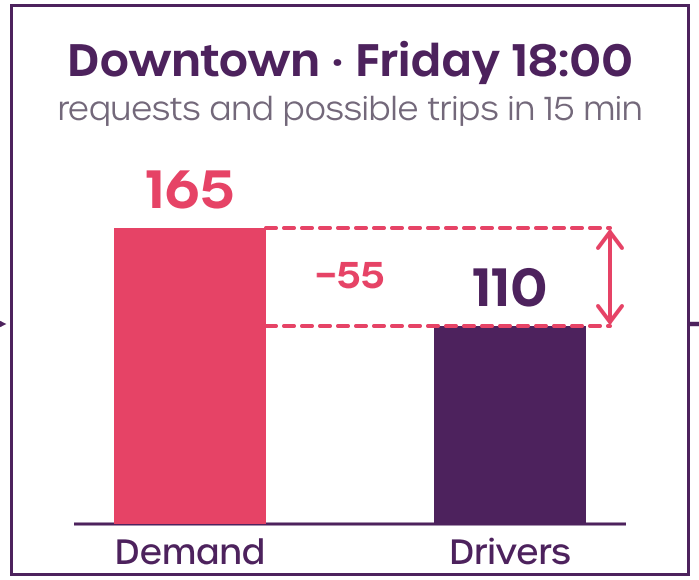
Uber: the problem

In every zone and every few minutes, demand and drivers almost never match

WHAT MOVES DEMAND

- Time and day
- Weather
- Events
- Traffic

THE ZONE RIGHT NOW



IF NOBODY ACTS

Demand above supply

Long waits, more cancellations and less satisfied users

WHAT HAPPENS IN DOWNTOWN

Supply above demand

Fewer trips per driver, less income and log-offs

IN ANOTHER ZONE OR AT ANOTHER TIME

Challenge: balance supply and demand within minutes, zone by zone

The challenge is not just to predict: it is to balance supply and demand before the gap hurts everyone.

Source: Shuaib Ullattil, Medium (5 Feb 2026)

Dynamic pricing and demand

Uber (global presence)

Explanatory case study; Uber's real models are proprietary

Concept

Predict demand per zone and adjust the price in real time.

Advantage

Shorter waits and a more stable driver supply.

Real use case

Time series, clustering and regression to set the dynamic price.

Limitation

Sharp surges feel unfair and require sensitive data.

The article publishes no real figures or architecture: the numbers in this deck are synthetic.

Price works as a signal that balances the system, not just as a way to charge more.

Source: Shuaib Ullattil, Medium (5 Feb 2026)

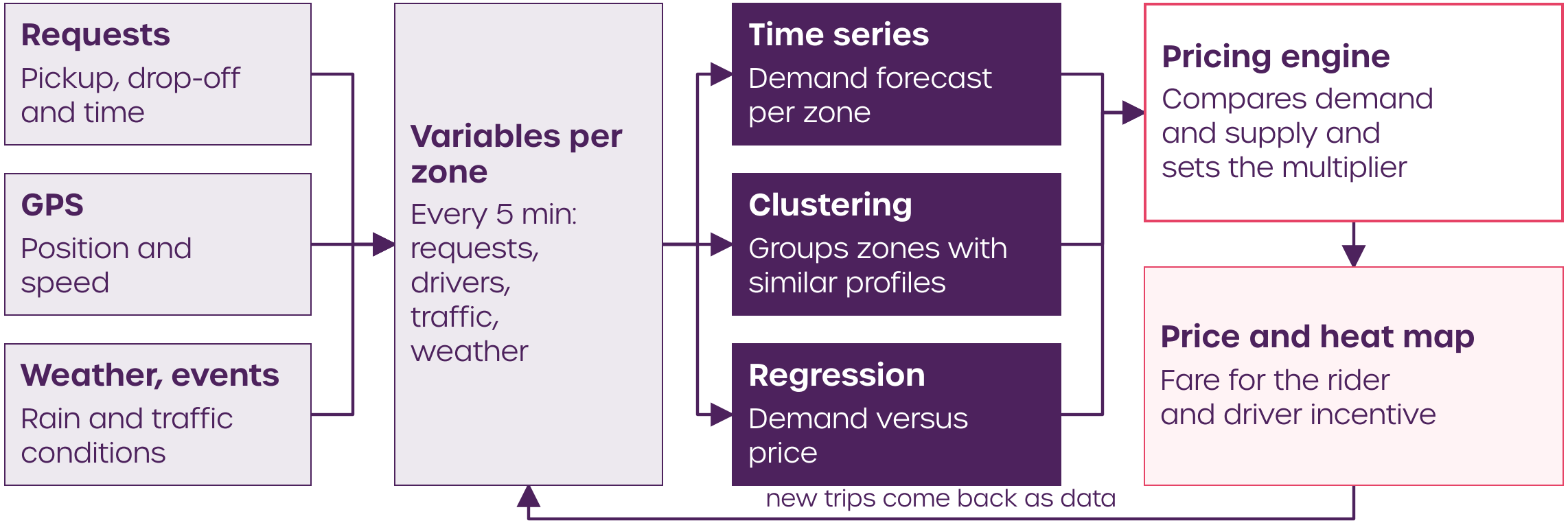
The starting data

Source	What it contains	What it is for
Requests and trips	Pickup, drop-off, time, duration, route, cancellations and payments	Measure real demand and its history
Phone GPS	Position and speed of each driver, every few seconds	Know supply and traffic in real time
External context	Weather, road closures and local events	Anticipate peaks that history does not explain
App usage	Ratings, cancellations and comments	Understand behaviour and satisfaction

Volume, variety and velocity: the three "V"s of Big Data. Source: Shuaib Ullattil, Medium (5 Feb 2026).

Uber: from data to price

From raw data to the price the rider sees, with a feedback loop



Didactic reconstruction based on the article's description: Uber's real architecture is not public.

Source: Shuaib Ullattil, Medium (5 Feb 2026)

Algorithms

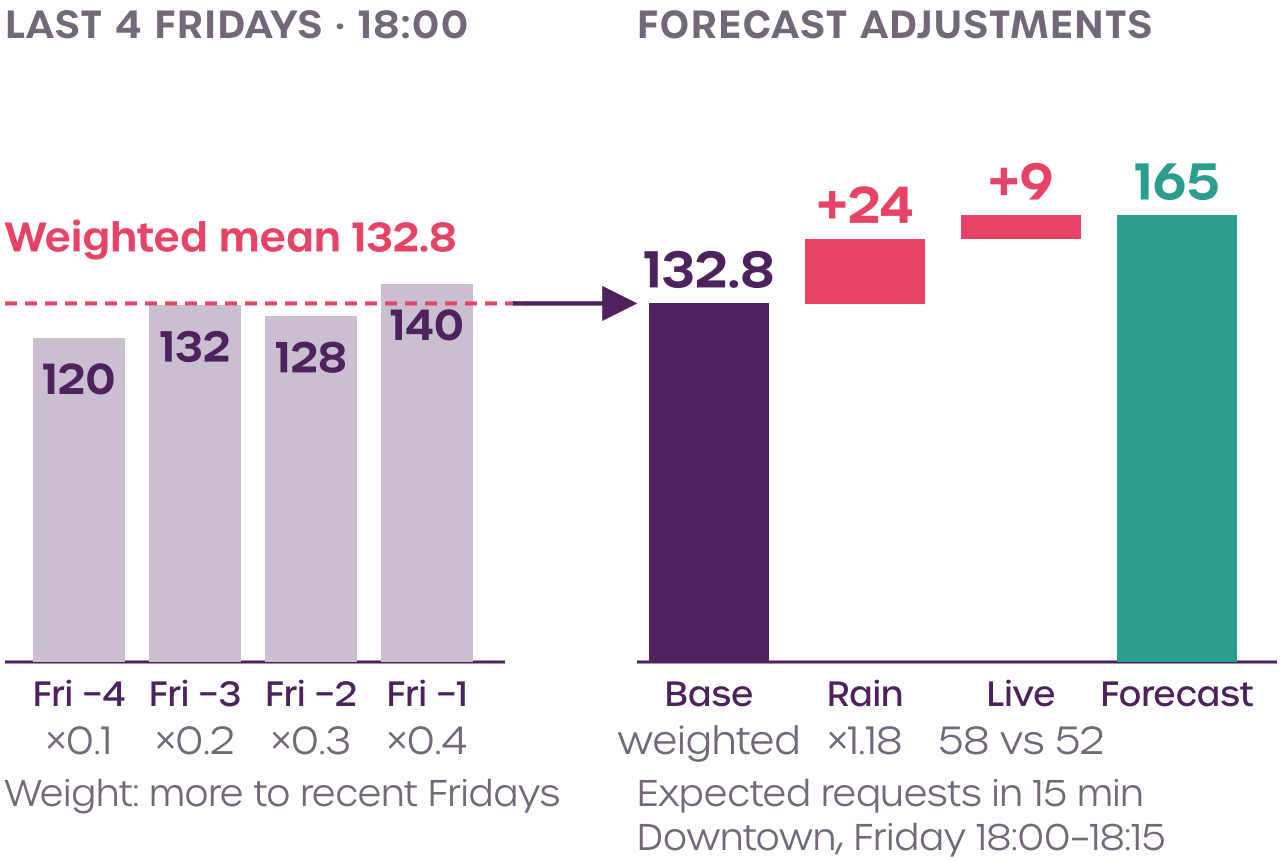
Four pieces, step by step

Each one with a synthetic numeric example: Downtown, Friday 18:00, with rain.

Time series: how many trips will there be?

Algorithm 1/4 Forecast of Downtown demand between 18:00 and 18:15

1. **Look at the past.** Requests at that time over the last 4 Fridays: 120, 132, 128 and 140.
2. **Weight.** Recent Fridays count more: 0.1·120 + 0.2·132 + 0.3·128 + 0.4·140 = **132.8**.
3. **Adjust for context.** A model learned that rain adds ≈18% demand: 132.8 × 1.18 = **156.7**.
4. **Correct live.** In the first 5 min there are 58 requests against 52 expected (+11%). Half is applied: 156.7 × 1.055 = **165**.



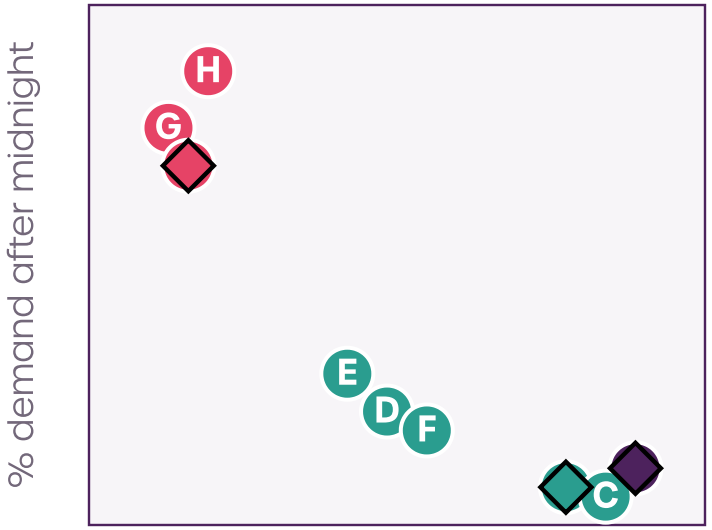
Synthetic data. Uber does not publish its models: a weighted seasonal average with adjustments is used here, the simplest form of a time series.

Innoit

Clustering: what kind of zone is it?

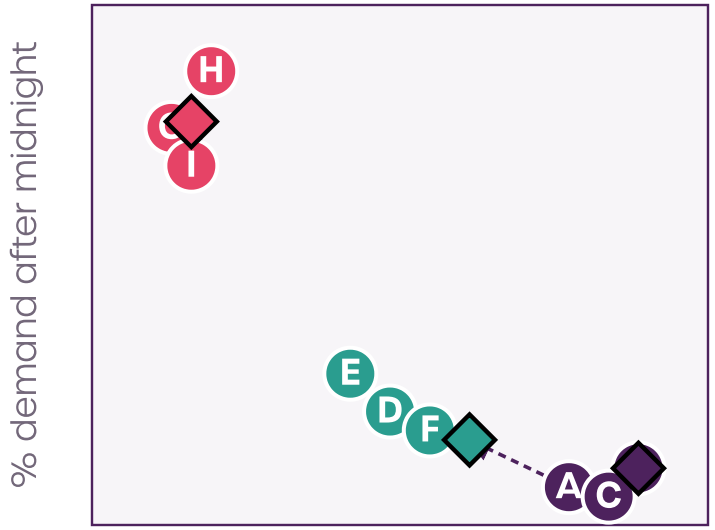
Algorithm 2/4 k-means groups zones by their demand profile (k = 3)

1 · Initial centres



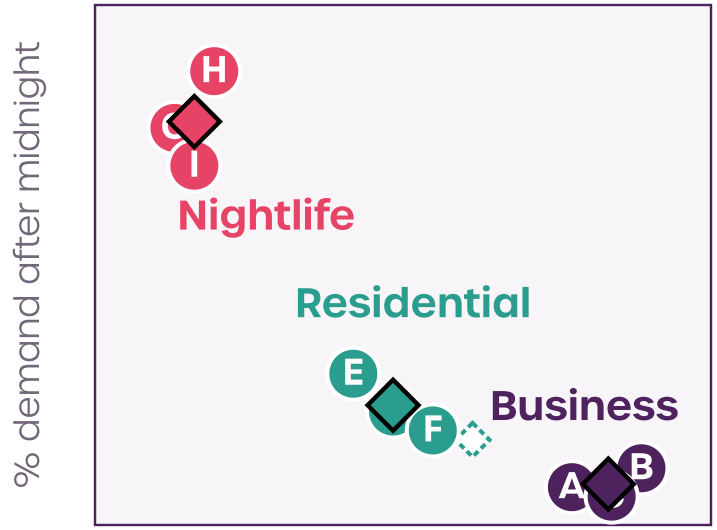
Two centres land in the same business zone. Each zone joins its nearest centre.

2 · Move and reassign



Each centre moves to the mean of its zones. A and C change group.

3 · Repeat until it converges



No zone changes: 3 groups remain. Centre (C) is a business zone.

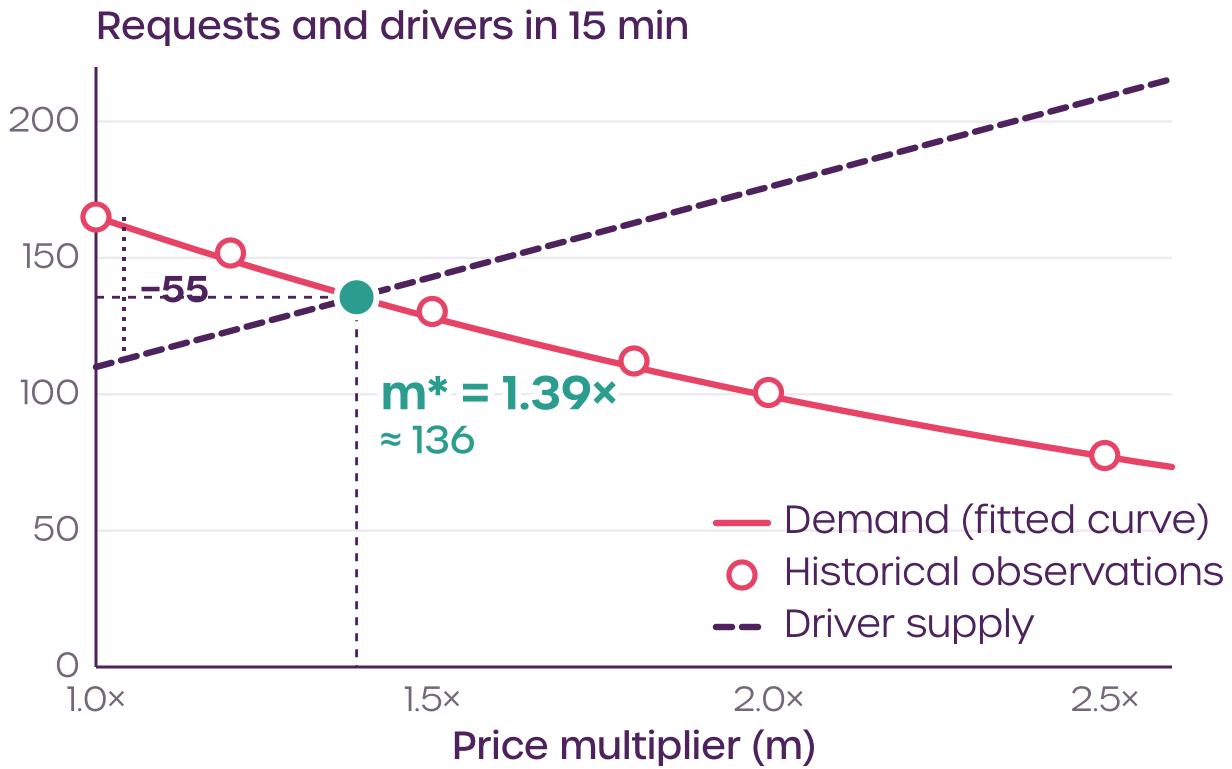
Synthetic data for nine zones (A–I). Each zone is described by two numbers; the groups emerge from the data, with no prior labels. Zones in the same group share a pattern: they are forecast together and it is anticipated where to send drivers.

Innoit

Regression: demand versus price

Algorithm 3/4 Sensitivity of demand to price and equilibrium price

1. **Gather (price, demand) pairs.** Six historical levels: at 1.0× demand is 100%; at 1.5×, 79%; at 2.5×, 47%.
2. **Choose the shape.** Demand falls exponentially: `demand(m) = base · e^(−β·(m−1))`.
3. **Fit by least squares** on ln(demand): **β = 0.51**, $R^2 = 0.999$. Each +1.0× removes ≈40% of demand.
4. **Find the equilibrium.** Demand(m) = supply(m): $165 \cdot e^{(-0.51(m-1))} = 110 \cdot (1 + 0.6(m-1)) \rightarrow$ **m* = 1.39×**, about 136 trips.



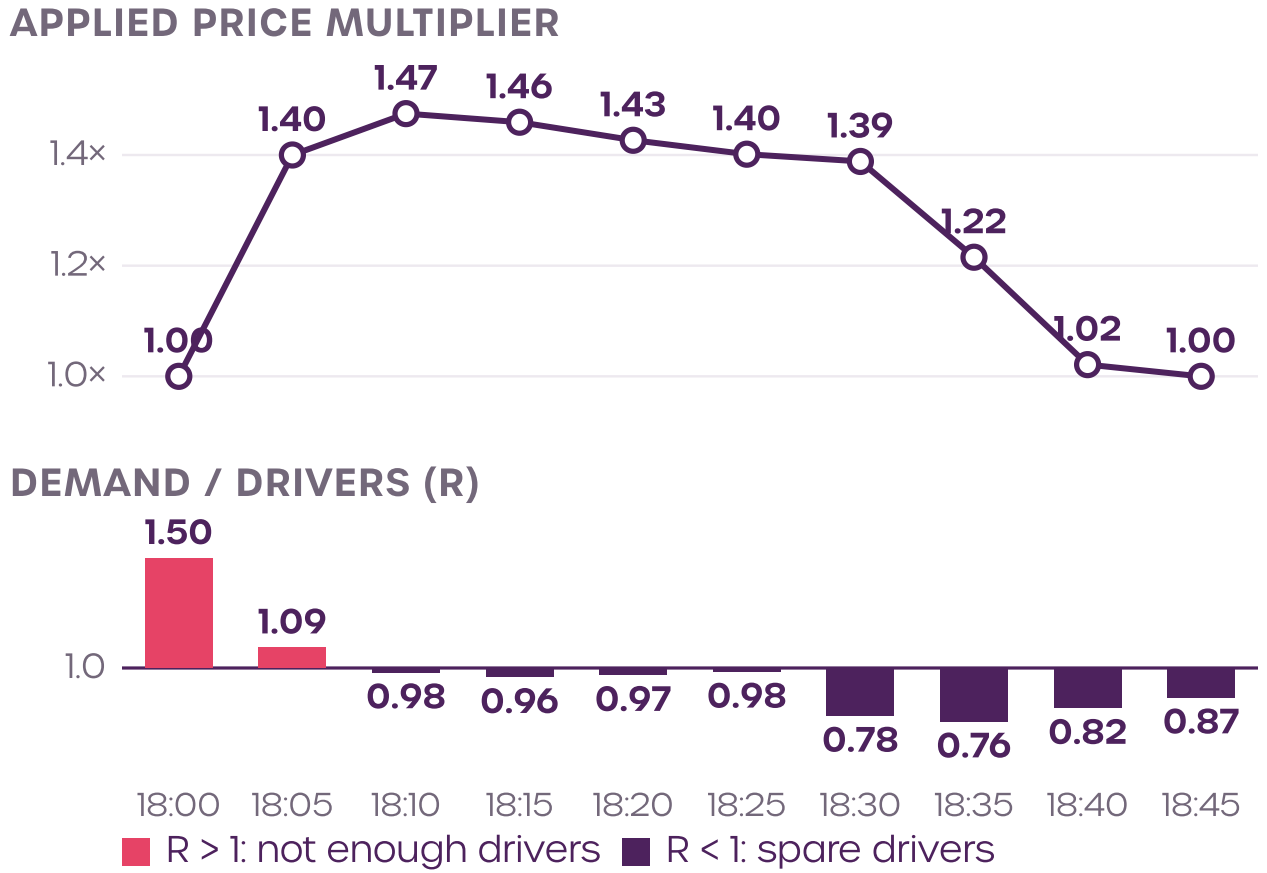
demand(m) = 165 · e^(−0.51·(m−1))
supply(m) = 110 · (1 + 0.6·(m−1))

Synthetic data. The supply response (+60% drivers for every +1.0×) is a didactic hypothesis; in practice it is also estimated from data.

Price loop: adjust every 5 minutes

Algorithm 4/4 Feedback control: the price is corrected every 5 minutes until it balances

1. **Measure.** Every 5 min: R = demand / drivers. At 18:00, 165 / 110 = **1.50**.
2. **Correct the price.** $m \leftarrow m + 0.8 \cdot (R - 1)$, between 1.0× and 3.0×: 1.00 + 0.8·0.50 = **1.40×** from 18:05.
3. **Let it react.** Some riders wait and drivers move closer. At 18:05, R = 1.09 → **1.47×**.
4. **Repeat and relax.** It settles at ≈1.4×. When demand falls (R < 1) the price drops to 1.00× at 18:45.



Synthetic simulation with the demand and supply of the example. When demand drops at 18:30, the multiplier returns to 1.00× on its own.

A Friday at 18:00, end to end

Downtown, Friday 18:00, with rain: the four algorithms chained with the example figures



Synthetic figures consistent with the previous slides: not Uber data.

Source: Shuaib Ullattil, Medium (5 Feb 2026)

Challenges and ethics

Privacy

Real-time locations of riders and drivers: they require consent, security and data governance.

Perception of fairness

Raising prices in emergencies or bad weather feels abusive, even if it balances the system.

Technical scale

Streams from millions of users with low latency; models are monitored and retrained.

Ideas to mitigate

A cap on the multiplier, a price shown before confirming and clear notices to the user.

Discussion

What price or priority do you adjust by hand today that a data loop could balance?

What would feel unfair to you even if it were efficient?

www.inno-it.es

Innoit